

FOR SOFTWARE ENGINEERS: THE CEILING NOBODY MENTIONS

Kafka Partitions

Why adding consumers stopped making it faster

Four consumers. Three partitions. The fourth is healthy, its logs are fine, and it is doing nothing at all.

Traffic is growing so you add consumers, and throughput does not move. The new instance is running, it is healthy, and it is idle.

That is not a bug and not a misconfiguration. It is Kafka behaving exactly as designed, and the reason is the single most useful thing to understand about scaling a consumer group.

Every Kafka statement below was checked against the Apache Kafka documentation named beside it, on the date at the foot of this sheet.

In scope

- Why the unit of work handed to a consumer is a partition rather than a message
- What the word GROUP is doing in "one partition per consumer"
- Why the idle consumer is capacity rather than waste
- What a rebalance costs, which is the part a senior answer includes
- Why partition count is an architecture decision and not a tuning knob

Out of scope

- Rebalance protocols and their tuning, which is episode 3
- Offset commit semantics, which is episode 4
- Any claim that a particular partition count is right for your system
- Broker sizing, which is an operations question rather than a design one

Start with offsets belonging to partitions. Every other line on this page is a consequence of that one fact.

01 Three partitions, and what each consumer count does

1 consumer	Holds all three partitions. No parallelism, full coverage
2 consumers	Two and one. Uneven, and still correct
3 consumers	One each. The ceiling, and the most throughput available
4 consumers	Three working, one idle. The idle one is standby capacity
A second GROUP	Its own independent assignment across the same three
More partitions	Raises the ceiling, and changes where existing keys land

Consumers beyond the partition count are not slower. They are unassigned. The ceiling is the partition count, and nothing about the consumer changes it.



03 Offsets belong to partitions, not topics

An offset is a position within ONE partition. It is not a position in a topic, because a topic has no single ordering to have a position in.

That one fact decides everything else on this page. If progress is tracked per partition, then the unit of work that can be handed to a consumer is a partition. Not a message, and not a topic.

Each partition is an ordered, immutable sequence of records, and the offset uniquely identifies each record within its partition.

Kafka 4.3, Topics and Logs

A position tracked per partition makes the partition the smallest unit that can be assigned without two consumers disagreeing about progress.

First principles, not vendor documentation

04 The limit is per consumer group

Inside a consumer group, Kafka assigns each partition to exactly one consumer. Never two. That is what makes the offset meaningful: if two consumers in one group read the same partition, neither could say where the group had reached.

So with three partitions a group can put at most three consumers to work. A fourth joins successfully, receives an empty assignment, and waits.

The word GROUP is doing real work in that sentence. A second consumer group gets its own independent assignment across the same three partitions. A payments processor and a fraud service can both read every message. Groups do not divide a topic between applications, they divide it between instances of the same application.

Each partition is consumed by exactly one consumer within each subscribing consumer group.

Kafka 4.3, Topics and Logs

Consumers in excess of the partition count receive no assignment rather than sharing one.

Kafka 4.3, Consumer Configs



05 The idle consumer is not waste

It is easy to look at a consumer with no partitions and see money being burned. It is standby capacity.

If an active consumer dies, Kafka rebalances and hands its partition to the idle one. No deploy, no alert anyone has to act on, no gap while somebody scales the group.

The fourth consumer is the reason the outage does not become an incident. That is the sentence almost nobody says, and it is the one worth saying.

When group membership changes, partitions are redistributed among the remaining members, so an unassigned member can receive work without any deployment.

Kafka 4.3, Consumer Configs

06 A rebalance is not free

When group membership changes, partitions are reassigned, and during that consumption pauses.

If your consumers hold state built from the partitions they own, that state has to be rebuilt somewhere else.

This is why "just add more consumers" is not a free action in a running system, and why a senior answer names the cost rather than only the mechanism.

A change in group membership triggers a reassignment during which the affected consumers are not processing records.

Kafka 4.3, Consumer Configs

Local state derived from an owned partition does not survive that partition moving to another instance.

Operational practice, not vendor documentation

07 So why not just create a hundred partitions

Because partition count is not an ordinary tuning knob. You can increase it. You can never decrease it.

And increasing it changes where existing keys are routed. The default partitioner for keyed records hashes the key against the number of partitions. Change the count and the same key can map somewhere different. A payment that lived in partition one may now land in partition three while its older events stay where they were.

Ordering is then broken across two partitions for exactly the keys that were in flight. Not corrupted, not lost, but no longer ordered, which for a payment system is the same class of problem.

That is why partition count is an architecture decision made early rather than a setting adjusted under load.

The number of partitions for a topic can be increased but not reduced.

Kafka 4.3, Topics and Logs

Partition selection for a keyed record depends on the current partition count, so increasing it redirects existing keys.

Kafka 4.3, Producer Configs

08 The answer an interviewer is listening for

Ordering per key, parallelism across keys.

Key by the entity that must stay ordered, which for payments is the payment itself. Every event for one payment lands in one partition and is processed in order. Different payments sit in different partitions and are processed at the same time.

You get both properties from the same choice, and the partition count sets the ceiling on the second one. Then add the cost: the ceiling cannot be lowered later, and raising it disturbs keys in flight.

Per-key ordering and cross-key parallelism are two consequences of one decision, which is what to use as the key.

First principles, not vendor documentation

Before you scale a consumer group

1. How many partitions does the topic have right now?
2. How many consumers are in the group, and how many are assigned?
3. Is the idle consumer deliberate standby, or an accident nobody costed?
4. Is this one group, or several applications sharing a group id by mistake?
5. What state do consumers hold that a rebalance would destroy?
6. How long does a rebalance pause consumption for, measured rather than assumed?
7. If we raise the partition count, which keys are in flight that day?
8. Who signs off partition count, and is it written down anywhere?
9. Can I demonstrate the idle consumer on my own machine?
10. Can I state the ceiling and why it cannot be lowered?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



x

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Apache Kafka and the Kafka logo are trademarks of the Apache Software Foundation. PostgreSQL and its logo are trademarks of the PostgreSQL Community Association. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-16 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.